# DOM-Guided tiling for visual document RAG

Anonymous ACL submission

## Abstract

Visual retrieval-augmented generation renders documents as images and lets a vision-language model read them directly, motivated by avoiding the losses of OCR and text extraction. Existing systems cut the rendered page on a fixed pixel grid, blind to content: a table can be split across tiles, and the retriever must recover boundaries the renderer already knew. On the web, this segmentation is free – the browser's layout engine exposes every element's bounding box before a single pixel is cropped. We introduce DOM-Graph RAG, which tiles pages along their own DOM elements, organizes the tiles into a multigranular evidence graph, and reads them with a contrastively fine-tuned VLM. Against a blind fixed-height grid, DOM-guided tiling reaches a page's content at a smaller total pixel budget, and our fine-tuned reader nearly doubles top-1 retrieval on held-out articles. We also report two negative results: under the same lexical scorer, flat text chunking is competitive with DOM tiles, and our budgeted graph controller does not yet beat static top-k on cross-page questions. The full pipeline is open-sourced.

## 1 Introduction

Retrieval-augmented generation (RAG) over visually rich documents – web pages, tables, infoboxes – has long relied on text extraction: parse the HTML or run OCR, chunk the text, retrieve with a dense retriever. This discards exactly what layout encodes: which numbers belong to which table row, which caption belongs to which figure. Visual RAG avoids this by having a vision-language model (VLM) retrieve directly over rendered page screenshots – PixelRAG (Wang et al., 2026) shows this can match or beat text-based retrieval on long, layout-heavy documents. But existing visual RAG systems cut the page on a fixed pixel grid, blind to content: every tile has the same height regardless of what is on the page, so a table row or paragraph can be split with no awareness it was ever one unit. A retriever built on such tiles must reconstruct boundaries the renderer already knew and threw away.

On the web, that reconstruction is unnecessary. For a rendered page, the browser's own layout engine already knows every element's exact bounding box before a single pixel is cropped – the segmentation a blind grid discards is sitting in the DOM, free. **DOM-Graph RAG** tiles pages along this structure instead, and organizes the resulting tiles into a multigranular evidence graph in the style of MAGE-RAG (Zuo et al., 2026) (§2), which reads an already-segmented long document under a budgeted controller but never has to obtain content-aligned tiles from a rendered page in the first place.

**Contributions.** We introduce an open-source pipeline[1] (§3):

1. *DOM-guided adaptive tiling*, replacing a blind pixel grid: elements cropped along their own bounding boxes, undersized ones merged, oversized ones split, headings paired with their content (§3.1). Ablation (a) (§5) shows it reaches a page's content at a substantially smaller pixel budget than a blind grid, at a higher answer-attachment rate.
2. a *multigranular graph* in the style of MAGE-RAG, instantiating all five of its relations plus a cross-page hyperlink relation and a `continues` relation for oversized pages split into sections (§3.2).

The pipeline also includes a contrastively fine-tuned VLM reader (§3.3), which nearly doubles held-out Recall@1 over the base model (ablation (b)), and a budgeted online evidence controller generalizing MAGE-RAG's controller to live-DOM graphs (§3.4), whose cross-document

[1] https://anonymous.4open.science/r/domgraph-rag-5886

graph expansion measurably narrows – but does not yet close – the gap to static top-$k$ retrieval on cross-page questions (ablation (c)).

The full pipeline is implemented and exercised end-to-end on live Wikipedia pages, with dependency-light unit tests over the geometric, graph, mining, and image-processing logic. §5 reports these ablations in full and lays out the evaluation against MAGE-RAG's published baselines as work in progress.

## 2 Related Work

**Text-based document RAG.** Standard RAG chunks extracted text and retrieves with a dense bi-encoder (Karpukhin et al., 2020). On visually rich documents this requires a layout-aware parser or OCR, both lossy on complex layouts, and discards 2D structure before retrieval ever runs.

**Screenshot-based visual RAG.** PixelRAG (Wang et al., 2026) retrieves over rendered-page image tiles with a VLM embedding model, part of an established line retrieving over page images rather than text: ColPali (Faysse et al., 2024) (late-interaction patch embeddings), Document Screenshot Embedding (Ma et al., 2024a) (whole-page VLM embeddings, no parsing), and VisRAG (Yu et al., 2024)/M3DocRAG (Cho et al., 2024) (retrieve-then-generate pipelines, the latter multi-page). None address how a page is cut into retrievable units for a VLM – what our tiling (§3.1) targets, so it could feed any of them, not only PixelRAG, whose own blind fixed-height grid lets a table or paragraph straddle a tile boundary. PixelRAG also reports up to 3× token-cost reduction at lower resolution without specifying the filter or resolutions tested; we build a matched full-/reduced-resolution pair on our own tiles to measure this directly (§3.3).

**DOM-based and block-level web page segmentation.** Cutting a page into coherent units predates VLM retrieval: VIPS (Cai et al., 2003) segments a page top-down, combining DOM structure with visual cues into a tree of content blocks, and block-based web search (Cai et al., 2004) shows indexing and query expansion at this block level improves web retrieval over the whole page. Our tiling (§3.1) targets the same problem with a different mechanism – boundaries come directly from each element's own bounding box plus merge/patch rules, not VIPS's recursive coherence heuristics – and for a different consumer: a multimodal embedding rather than block-level lexical features.

**Graph-structured evidence and controllers.** MAGE-RAG (Zuo et al., 2026) (evaluated on LongDocURL (Deng et al., 2025), MMLongBench-Doc (Ma et al., 2024b)) builds an offline multigranular evidence graph and reads it with a budgeted controller – the direct ancestor of §3.4. The difference is provenance: we build the graph from a rendered page's DOM, giving exact boundaries and reading order for free, and instantiate all five of its relation types (§3.2). GraphRAG (Edge et al., 2024) shares the premise of building an explicit graph before querying it, over extracted entities rather than layout structure. Layout- and markup-aware pretraining (LayoutLM (Xu et al., 2020), MarkupLM (Li et al., 2022)) instead bakes structure into the model's representation; we use the DOM directly at inference time, with no pretraining required. Web-agent work (WebArena (Zhou et al., 2023), Mind2Web (Deng et al., 2023), SeeAct (Zheng et al., 2024)) has already studied when DOM structure is reliable enough to act on – the same failure mode (JS-hydrated, div-soup markup) that scopes our own claim to semantically marked-up pages (§6).

**Contrastive fine-tuning and hard-negative mining.** Training with in-batch negatives plus mined hard negatives is standard in dense retrieval, e.g. DPR's BM25-mined negatives (Karpukhin et al., 2020); we use TF-IDF at tile granularity instead. Training pairs are generated synthetically by prompting a VLM per tile, in the spirit of InPars (Bonifacio et al., 2022) and Promptagator (Dai et al., 2022) adapted to image tiles. We fine-tune with LoRA (Hu et al., 2021) on both the decoder and the vision tower.

## 3 Method

A page is rendered once with a headless browser (Figure 2); its DOM geometry drives an adaptive tiling step (§3.1); tiles are organized into a two-level graph, optionally spanning several hyperlinked pages (§3.2); a VLM reader is contrastively fine-tuned to score (question, tile) pairs (§3.3); and, at query time, an online controller walks the graph under a budget to decide which nodes the reader actually opens (§3.4).

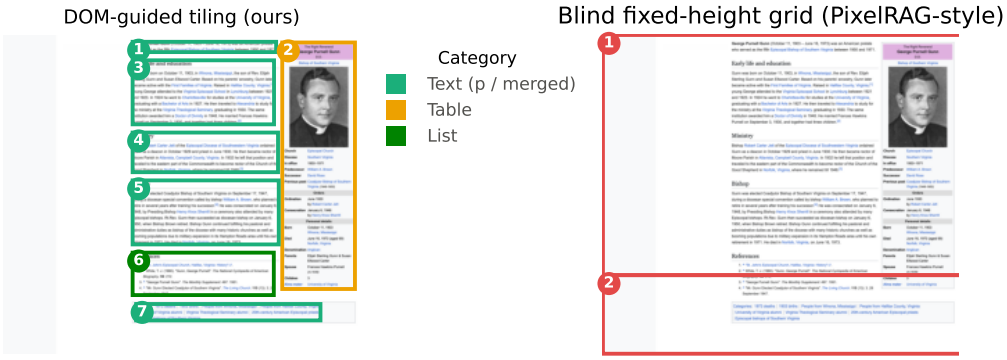


Figure 1: Same page (`en.wikipedia.org/wiki/George_P._Gunn`), tiled two ways. **Left, DOM-guided (ours):** boundaries follow the page's own elements – the infobox is one tile, body paragraphs and the reference list are separate, headings pair with their content. **Right, blind fixed-height grid (PixelRAG-style):** bands are cut by pixel height alone, with no notion of what they contain – here nearly the entire page collapses into a single undifferentiated tile, infobox and body text alike. Ablation (a) (§5) quantifies the consequence: DOM tiling reaches this content at a 2.7× smaller total pixel budget.

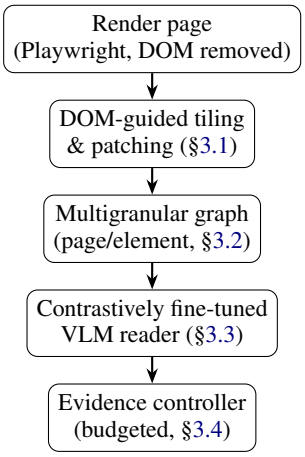


Figure 2: Pipeline overview. Rendering, tiling, and graph construction happen once per page; the reader is fine-tuned once, offline; the evidence controller runs once per query, deciding which graph nodes the reader opens.

### 3.1 DOM-Guided Adaptive Tiling

**Rendering and extraction.** We load the page in headless Chromium at a fixed viewport width (2048 px), remove interface chrome via a CSS selector list, and take a full-page screenshot. For a configurable tag set (`h1`–`h4`, `p`, `table`, `figure`, `img`, `ul`/`ol`, `blockquote`, `pre`) we read each element's absolute bounding box from the rendered DOM. For non-text containers this is `getBoundingClientRect`; for text-flow tags, whose CSS box spans the full container width regardless of actual content, we instead union the client rects of every descendant text node. Without this, a paragraph or reference list sharing a row with a floated infobox reports a bounding box running through the infobox's column even though its text stops well short of it. Hyperlinks are collected from `<p>` elements only, seeding the cross-page edges of §3.2.

**Overlap resolution.** Nested tags (an `<img>` in a `<figure>`) yield duplicate bounding boxes for the same content; we sort by DOM depth and drop any element already fully contained (1 px tolerance) in a shallower kept element. A floated sidebar and the text sharing its row can also genuinely intersect even after the text-flow tightening above. Rather than tile such elements independently and duplicate the shared pixels, we connect any two elements whose bounding-box intersection exceeds a small fraction of the smaller one's area, and tile each connected component as one crop spanning their union.

**Adaptive tiling and patching.** Elements are processed top-to-bottom under three rules: (1) *heading pairing* – a heading is never a standalone tile, but merged with the next element (e.g. `h2+p`); (2) *merging* – non-heading elements under 80 px are buffered until their union spans that minimum, then flushed as one `merged` tile; (3) *patching* – an element taller than 2048 px is sliced into $\lceil h/h_{\max} \rceil$ equal-height sub-tiles. This targets PixelRAG's failure mode directly: a table that would straddle a fixed grid cell is instead one tile, and a short caption merges with its neighbor rather than being orphaned. Each tile keeps a pointer to its

source element(s), used both for the graph (§3.2) and as ground truth for negative-mining (§3.3). Figure 4 (Appendix E) shows tiles overlaid on a real page.

### 3.2 Multigranular Graph Construction

Tiles are organized into a two-level directed graph (one `page` node, one `element` node per tile) with all five of MAGE-RAG's relation types, plus one of our own for cross-page connectivity:

- **`contains`** (page → element): every element belongs to its page.
- **`reading_order`** ($\text{element}_i \rightarrow \text{element}_{i+1}$): a spatial heuristic, *not* DOM order – elements are grouped into visual rows (20 px vertical-overlap tolerance), each sorted left-to-right, rows concatenated top-to-bottom, since DOM and visual order diverge under CSS positioning.
- **`layout_adjacency`** (element ↔ element): reciprocal edges to the 2D neighbors reading order flattens away (left/right within a row; above/below to the nearest horizontally-overlapping tile in the next row).
- **`section_hierarchy`** (heading → child): each heading points to its nested sub-heading/content, forming a section tree distinct from the flat `contains` relation.
- **`semantic_neighbor`** (element ↔ element): TF-IDF-cosine edges between lexically close elements regardless of position, added greedily under a per-tile degree cap. Optional, off by default, meant to be toggled to compare the controller with and without it (§5).
- **`links_to`** (element → external_page): hyperlinks from the first $k$=3 link-bearing text tiles (reading order); beyond that budget, links are dropped to bound the corpus crawl's branching factor.

Every element node starts with a `state` attribute (INACTIVE), which the controller (§3.4) evolves at query time. Figure 5 (Appendix E) shows the resulting graph for the page of Figure 4.

**Oversized-page splitting.** One article's screenshot exceeded 46,000 px, producing dozens of tiles whose sequential QA generation (§3.3) ran for hours with no partial progress saved on failure. Rather than exclude such pages, one whose element height exceeds 10× the max tile height is split into sections *before* tiling, at `h1`/`h2` boundaries, with any still-oversized section further rechunked by a height budget. Each section is tiled and graphed as a standalone page would be; the per-section graphs are merged by node-namespacing, with consecutive sections' `page` nodes linked by a new **`continues`** relation – sequential parts of the same document, distinct from the hyperlink relation `links_to`. Every page's graph, split or not, exposes an *entry point* for the corpus graph below to rewire onto. Figure 6 (Appendix E) shows an example split into 16 sections.

**Corpus graph.** From a seed page we crawl up to $m$ pages reachable via its `links_to` edges (one hop, non-recursive), build each linked page's graph independently, and merge by namespacing every node with a URL-derived slug, rewiring each placeholder external-page node onto the crawled page's entry point. `links_to` is the only cross-document connective tissue, `continues` the only cross-section one – the substrate the evidence controller (§3.4) operates over.

**Structured rendering.** Any (sub-)graph serializes its page node and directly-contained elements to XML (one `<element id=... tag=... state=...>text</element>` per node) as the reader's structured input, in the spirit of MAGE-RAG's structured-rendering step (example in Appendix E, Figure 7).

### 3.3 Contrastive Reader Fine-Tuning

To turn a general-purpose VLM into a tile-level reader that also scores how well a tile answers a query, we fine-tune it contrastively on synthetic data, in three stages.

**Synthetic QA generation.** For every tile, we prompt the VLM with the tile image alone for one self-contained, factual question answerable *only* from that tile, or `SKIP` if it carries no exploitable content – the recipe of synthetic query generation for retriever training (InPars, Promptagator; Bonifacio et al., 2022; Dai et al., 2022) applied to image tiles. Every tile crop is persisted regardless of outcome, so `SKIP`ped tiles remain available as candidate hard negatives.

**Hard-negative mining.** For each question, we rank every other tile from the same page by TF-IDF cosine similarity and keep the top $n$=2 clearing two false-negative filters: (a) a lexical filter rejecting any tile whose text already contains the an-

swer verbatim, and (b), optionally, a VLM-judge filter showing the candidate image to the reader VLM and asking whether it can recover the answer there, catching non-lexical false negatives at the cost of one extra VLM call per candidate.

**Contrastive LoRA fine-tuning.** `Qwen2-VL` is generative, not a similarity encoder; we turn it into one via PromptEOL (Jiang et al., 2024) (a short fixed instruction, "summarize the above in one word," taking the last token's final-layer hidden state as embedding), originally proposed for text-only LLMs and applied here to a VLM's image+instruction input. LoRA ($r$=16, $\alpha$=32, dropout 0.05) adapts both the decoder's attention projections and the vision tower's, so the model can refine what it attends to in the image jointly with how it reads the question. Given $B$ questions each with one positive and $K$ mined negatives, we minimize a temperature-scaled ($\tau$=0.05) InfoNCE loss over the $B(1{+}K)$ tiles in the batch, so each positive competes against its own negatives and, in-batch, every other question's tiles.

**Resolution as an efficiency lever.** PixelRAG reports up to 3× token-cost reduction at lower resolution without specifying the filter or resolutions tested. We build the infrastructure to test this directly: every tile can be mirrored via Lanczos resampling at a configurable scale (0.5 default), reusing the same questions/positives/negatives at both resolutions so comparing them is a matter of pointing fine-tuning at one image root or the other (§5).

### 3.4 Online Evidence Controller

Given a query, the controller decides which element nodes the reader actually reads, rather than the whole page or a static top-$k$. It evolves each node's `state` through a four-state machine (INACTIVE→ACTIVE→OPENED/PRUNED, Appendix E, Figure 3), generalizing MAGE-RAG's Algorithm 1.

**Relevance scoring.** Every element node is scored against the query by TF-IDF cosine similarity (same mechanism as hard-negative mining), a deliberate placeholder for the fine-tuned reader's own learned similarity (§6).

**Policy.** Algorithm 1: (1) seed the frontier with the $k$ highest-scoring inactive nodes; (2) repeatedly take the highest-scoring *active* node, pruning it for free below an opening threshold or opening it (one unit of budget $B$) and activating still-inactive neighbors that clear an activation threshold; (3) once budget is exhausted or no active node remains, prune whatever is left active. Neighbors are only considered *after* their node is opened, conditioning the frontier on what has already proven relevant rather than a fixed global ranking – what distinguishes the controller from static top-$k$ retrieval over the same scores (ablation (c)). "Neighbor" includes `links_to`/`continues` targets, not just same-page `reading_order`: the fix that lets the frontier reach a second document at all, instead of only through the initial seed (§5).

**Algorithm 1** Budgeted best-first evidence control

**Require:** graph $G$, query $q$, budget $B$, seed count $k$, thresholds $\theta_{\text{open}}, \theta_{\text{act}}$

score every element node by $\text{sim}(q, \text{text}(n))$
activate the $k$ highest-scoring inactive nodes ▷ seed
$b \leftarrow 0$
**while** $b < B$ and some node is ACTIVE **do**
  $n \leftarrow \arg\max$ score over ACTIVE nodes
  **if** $\text{score}(n) < \theta_{\text{open}}$ **then**
    $n \leftarrow$ PRUNED ▷ free
  **else**
    $n \leftarrow$ OPENED; $b \leftarrow b + 1$; record its text as evidence
    **for** each inactive neighbor $n'$ of $n$ via `reading_order`, `links_to`, or `continues` **do**
      **if** $\text{score}(n') \geq \theta_{\text{act}}$ **then**
        $n' \leftarrow$ ACTIVE
      **end if**
    **end for**
  **end if**
**end while**
prune every node still ACTIVE ▷ finalize
**return** concatenation of OPENED nodes' text, in reading order

**Output.** The concatenation of OPENED nodes' text, in reading order, is the final evidence passed to the downstream reader, with the controller's action log available for inspection.

## 4 Implementation

The pipeline is Python, one module per stage of §3: rendering/extraction via Playwright, DOM-guided tiling, graph construction with `networkx`, a cached VLM client (Ollama's `qwen3-vl` by de-

fault, or `Qwen2-VL-7B-Instruct` for LoRA fine-tuning), synthetic QA generation, TF-IDF hard-negative mining, and contrastive LoRA fine-tuning with `peft`/`torch`. Page fetching, tiling, and graph composition are checkpointed per page unit, so a failure partway through a long page only costs the section in flight. Dependency-light unit tests cover the geometric, graph, mining, and image-processing logic on synthetic fixtures; Playwright-backed tests check DOM extraction end-to-end against a bundled HTML fixture. Appendix A gives the full module-level breakdown.

## 5 Evaluation Protocol and Status

*The pipeline of §3 runs end-to-end on live pages and the trained reader is now evaluated held-out, but the full benchmark suite against MAGE-RAG's own baselines is not yet in place. We report three preliminary ablations below.*

**Benchmarks.** MAGE-RAG's protocol evaluates on LongDocURL (Deng et al., 2025) and MMLongBench-Doc (Ma et al., 2024b), but both are PDF benchmarks, and every input our method needs (DOM bounding boxes, CSS-selector chrome removal, `h1`/`h2` splitting) presupposes a renderable DOM a PDF does not have. We instead plan an HTML-native source: WebSRC (Chen et al., 2021) or HotpotQA (Yang et al., 2018), whose `supporting_facts` name the exact sentences, in two articles, needed per question – multi-hop by construction and the most direct fix for our own synthetic QA's single-hop bias (§6). We have implemented and scaled the HotpotQA path (`src/multihop_qa.py`, §6): 91/150 sampled questions (61%) yielded a complete cross-page example, used directly in ablation (c) below.

**Ablation (a): DOM-guided tiling vs. a blind fixed-height grid vs. flat text chunks.** The lexical scorer already used for hard-negative mining and controller relevance (§3.3, §3.4) lets us run this ablation without waiting on the trained reader. `scripts/grid_baseline.py` reimplements a PixelRAG-style blind grid (1,024 px bands, PixelRAG's own reported tile height, at our render width); a third condition chunks the same page region's extracted text into flat 400-character windows, the standard RAG baseline our introduction argues against but had not, until this revision, measured directly. `scripts/dom_vs_grid_experiment.py` re-renders 18 single-unit pages, builds all three, and ranks every existing `qa_pairs.jsonl` question against each by TF-IDF cosine, same scorer throughout. A question is scored in a condition only if its answer string occurs in one of that condition's units, else dropped from that condition rather than counted as a miss.

| | $n$ | R@1 | Attach | Pctl. rank |
|---|---|---|---|---|
| DOM | 213 | 0.540 | 52.1% | 0.868 |
| Grid | 109 | 0.716 | 26.7% | 0.833 |
| Text | 232 | 0.612 | 56.7% | 0.867 |

Table 1: DOM tiling vs. blind grid vs. flat text chunks, same TF-IDF scorer, 18 pages / 409 questions. Attach = share of the 409 questions whose answer string survives that condition's units at all (denominator for $n$ and R@1). Percentile rank $= 1-(\text{rank}-1)/(n_{\text{items}}-1)$ of the answer-bearing unit (1 = best, 0.5 in expectation under random ranking, regardless of pool size) – unlike raw Recall/MRR, not mechanically inflated by grid's $\sim 5\times$ smaller pool (Appendix D).

Two findings, not one. DOM tiling clearly beats the blind grid it targets: a higher attach rate (52.1% vs. 26.7%) at a **2.7× smaller total pixel budget per page** (Appendix D) – more of a page's content survives into a retrievable unit, more cheaply, once boundaries follow the DOM instead of a fixed height. But once pool size is corrected for (percentile rank), flat text chunking is lexically *competitive* with DOM tiling under this same scorer (0.867 vs. 0.868) and has the highest attach rate of the three. This is expected, not damaging: the ablation only ever scores extracted text, so it cannot show what a VLM gains from reading a content-aligned *image* tile over a same-size crop of raw text – that is what the fine-tuned reader of ablation (b) is for. What it does establish is pixel efficiency: DOM tiling reaches a page's content on a smaller image budget than a blind grid, without the retrieval cost a naive un-normalized comparison would suggest.

Two caveats on reading Table 1 directly. First, the grid and DOM attach rates are not measuring the same failure mode: a fixed-height grid *partitions* the page (every pixel belongs to exactly one band), so its lower attach rate reflects real content loss (an answer split across a band boundary); DOM tiles follow elements rather than tiling the canvas exhaustively and so do not partition the page (small gaps between bounding boxes belong to no tile), meaning a fraction of its own attach loss comes from a different, non-comparable source.

| | R@1 | R@3 | R@5 | MRR |
|---|---|---|---|---|
| TF-IDF (lexical) | 0.461 | 0.618 | 0.724 | 0.579 |
| Base (no LoRA) | 0.433 | 0.645 | 0.752 | 0.578 |
| LoRA fine-tuned | 0.785 | 0.915 | 0.952 | 0.856 |

Table 2: Base vs. LoRA-fine-tuned reader, held-out val split (330 questions, article-level, never seen in training; 8 dropped: page with <2 tiles or positive tile absent from the manifest); TF-IDF is the same lexical scorer used for hard-negative mining and controller relevance (`scripts/eval_retrieval_tfidf.py`), on the identical split and candidate pool. The untrained base VLM is essentially at the lexical floor (MRR 0.578 vs. 0.579); fine-tuning, not the VLM itself, is what buys the gain.

| Seed width $k_{\text{seed}}$ | Controller | Static top-$k$ |
|---|---|---|
| 3 (default; budget 5) | 39.6% (36/91) [30.1, 49.8] | 40.7% (37/91) [31.1, 50.9] |
| 5 (= budget) | 40.7% (37/91) | 40.7% (37/91) |

Table 3: Fraction of 91 cross-page multi-hop questions (§6) where the selected evidence covers both cited articles' positive tiles, controller with cross-document expansion (`links_to`/`continues`, §3.4) vs. a static top-$k$ baseline on the same TF-IDF scores; bracketed values are 95% Wilson intervals. Before this expansion (`reading_order` only), the same protocol gave 29.7% at $k_{\text{seed}}$=3.

The two attach rates should not be read as the same quantity measured twice. Second, $n$ differs by column (213, 109, 232) because a question is scored in a condition only if its answer string occurs in that condition's own units; each column's R@1 and attach rate is therefore computed over a different population of questions, not the same 409 questions three times over, and the table's rows should be compared with that in mind rather than as three scores on one fixed set.

**Ablation (b): fine-tuned reader vs. base model, held-out retrieval.** We added an article-level train/val split (`split_examples_by_article`, 80/20, disjoint articles in each split so no tile or near-duplicate question leaks across) and retrained LoRA on the train split only, on rented GPU compute (3 epochs, batch size 4, gradient checkpointing). `scripts/eval_retrieval.py` then scores every val-split question against *every* tile of its own page (a realistic retrieval pool, not the small mined positive/negatives set the training loss sees), with and without the adapter loaded on the same base model (`peft`'s `disable_adapter()`, so both passes share one model in memory).

Recall@1 nearly doubles (0.433 → 0.785) and MRR rises from 0.578 to 0.856 – a clean, held-out signal that the contrastive objective transfers to realistic per-page retrieval, not just the small mined pool it was trained against. The TF-IDF row locates that gain precisely: the untrained base VLM is essentially at the lexical floor (MRR 0.578 vs. 0.579), so the improvement comes from contrastive fine-tuning, not merely from using a VLM instead of a lexical scorer. This motivates wiring this reader into the controller's relevance scoring in place of TF-IDF (§6), left as the next step.

**Ablation (c): evidence controller vs. static top-$k$, cross-page multi-hop retrieval.** Method (§3.4) motivates the controller against the static top-$k$ baseline it generalizes – untestable on single-hop synthetic QA, where one tile always suffices, so we use the 91 HotpotQA-derived multi-hop questions of §6 instead. For each, we build the combined two-page graph of both cited articles, rewiring any `links_to` edge whose target is the other cited article onto its real entry point (as a genuine corpus crawl would, §3.2); run the controller and, on the identical TF-IDF scores, a static top-$k$ baseline; and check whether the evidence covers a positive tile on *each* page. An earlier version of the controller activated neighbors only along `reading_order`, which never crosses a page boundary, reaching a second document only through its literal seed – exactly what static top-$k$ already does, and worse whenever the seed alone missed one article; §3.4's policy now also activates neighbors via `links_to` and `continues`, closing that gap architecturally.

Cross-document expansion moves the controller from 29.7% to 39.6% at $k_{\text{seed}}$=3 – a tie within noise, no superiority claimed: the 95% Wilson intervals ([30.1, 49.8] vs. [31.1, 50.9]) overlap almost entirely, so this reads as statistical indistinguishability from the baseline, not a demonstrated improvement – and the two are exactly equal at $k_{\text{seed}}$=5=budget. The controller does not beat the static baseline at either seed width: once a second page is reachable at all, both policies tend to open the same highest-scoring candidates on it, so parity rather than a win is the expected outcome on this coverage metric – the controller's advantage over static top-$k$ is that it spends budget adaptively

| Statistic | Value |
|---|---|
| Distinct Wikipedia articles | 19 |
| Page units (articles + split sections) | 185 |
| Tiles extracted | 1,784 |
| QA pairs generated | 1,717 |
| Tiles with no usable QA (`SKIP`) | 67 (3.8%) |
| Contrastive examples (QA + hard negatives) | 1,339 |

Table 4: Contrastive dataset size (§3.3), lexical filter only (no VLM judge). 12 of 19 articles were split into sections (§3.2).

(pruning low-scoring seeds for free, propagating further where the page is more clearly relevant), not that it reaches structurally more content once both can reach it. We read this as a genuine fix to a real limitation (the graph's relations are now actually consumed, not just built and visualized, §6), not yet a positive result for the controller over the simpler baseline on this specific metric.

**Status.** All stages (§3) are implemented, tested, and demonstrated end-to-end on live pages (§4). QA-pair generation has scaled to the multi-page corpus of Table 4. Scaling surfaced one cost worth flagging: ∼2 minutes/tile for local QA generation, now partly mitigated by concurrent VLM calls (§4); Appendix C turns this into a compute-budget estimate for fine-tuning itself. A resolution ablation (reduced-scale tiles, §3.3) still requires a further training run on compressed tiles and is left as future work.

## 6 Limitations

**Controller expansion, fixed but not yet ahead.** All five of MAGE-RAG's relation types are instantiated (§3.2); the controller's frontier expansion (Algorithm 1) originally followed only `reading_order`, which never crosses a page boundary, and ablation (c) (§5) measured the resulting gap against static top-$k$ on cross-page questions. Adding `links_to`/`continues` expansion closes most of it (29.7% → 39.6% at default seed width) but the controller still does not strictly beat the baseline: `layout_adjacency` and `section_hierarchy` remain unconsumed, and `semantic_neighbor` is off by default.

**Single-hop self-generated data cannot exercise the graph – addressed via HotpotQA.** §3.3's QA generation asks for a question answerable only from the tile shown, so an oracle controller could answer every such question from one node. `src/multihop_qa.py` closes this gap with HotpotQA (Yang et al., 2018) instead: each question's `supporting_facts` names two Wikipedia articles and the sentences in each needed to answer it, so we render both, keep tiles overlapping a supporting sentence as positives, and mine hard negatives by TF-IDF pooled across both articles' tiles. Scaled to 91/150 sampled questions (61%) and used in ablation (c). Using this data for training, rather than evaluation only, remains future work.

**Chrome removal is Wikipedia-specific; tiling itself is not.** All quantitative results here are on Wikipedia, and chrome removal (§3.1) uses a MediaWiki-specific selector list. We rendered two non-Wikipedia pages (Python's documentation, MDN) as a qualitative check: the tiling mechanism itself (bounding boxes, merging, patching) produces coherent tiles over each page's article body, but each site's own navigation/sidebar chrome survives untiled, since it matches no selector we remove – what is Wikipedia-specific is the chrome list, not the tiling logic, though we make no quantitative claim beyond this observation. JS-hydrated SPAs and `div`-soup layouts remain out of scope, the regime web-agent work such as WebArena (Zhou et al., 2023), Mind2Web (Deng et al., 2023), and SeeAct (Zheng et al., 2024) already studies from the acting side.

**Other gaps.** No result above is at the level of the final *answer*: ablations (a)/(b) score tile retrieval (ranking, R@1/MRR) and ablation (c) scores evidence coverage (whether the right tiles were opened), never whether the pipeline's synthesized answer is actually correct end-to-end. `scripts/eval_multihop_e2e.py` implements this (evidence controller output fed to the reader for answer synthesis, scored by EM/F1 against HotpotQA gold answers) but is not yet reported here, pending the reader-in-the-loop controller fix below. Relatedly, ablation (b)'s gain has no external anchor yet: synthetic questions, the optional false-negative judge, and the fine-tuned reader all share the `qwen3-vl`/`Qwen2-VL` family, so the held-out Recall@1 improvement is entirely on our own model-generated questions; `scripts/eval_hotpotqa_reader.py` reruns it against this section's human-authored HotpotQA questions instead, implemented but not yet run to completion. Patching (§3.1) splits an

oversized element into equal-height sub-tiles regardless of content, a narrower version of PixelRAG's own failure mode. The controller's TF-IDF relevance scoring (§3.4) misses paraphrases and cross-modal cues the fine-tuned reader would catch; wiring in the reader's own similarity is the natural next iteration. Hard-negative mining is single-page outside HotpotQA; the corpus crawl is one hop, non-recursive; hyperparameters (Appendix B) are chosen once, not swept.

## 7 Conclusion

On the web, the segmentation a blind pixel grid discards is already sitting in the DOM: **DOM-Graph RAG** tiles pages along their own elements instead, organizes them into a multigranular evidence graph, and reads them with a contrastively fine-tuned VLM and a budgeted online controller. Every stage runs end-to-end on live pages. Against a blind grid, DOM-guided tiling reaches a page's content at a substantially smaller pixel budget and a higher answer-attachment rate, though a flat text-chunking baseline is lexically competitive under the same scorer – expected, since that scorer never sees the image a VLM would read (§5). The fine-tuned reader nearly doubles held-out Recall@1 over the base model. The controller's cross-document expansion (`links_to`/`continues`) closes most of a gap to static top-$k$ on cross-page questions that its earlier, `reading_order`-only version left wide open, though it does not yet strictly beat that baseline. Next: an HTML-native benchmark, the resolution ablation, and wiring the controller's relevance scoring to the now-validated reader in place of TF-IDF.

## Ethics Statement

The corpus builder fetches a seed Wikipedia page plus a small, bounded number of linked pages, with retry-and-backoff rather than aggressive refetching; a production deployment should prefer the Wikimedia REST API or a static dump over live scraping. Wikipedia text is CC BY-SA; embedded images carry heterogeneous, some non-free, licenses, so the screenshots/tiles produced are derivative works we do not release. Synthetic questions (§3.3) are VLM-produced and unverified, and should be labeled as such if released. No human subjects or user study.

## References


Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. 2022. Inpars: Data augmentation for information retrieval using large language models. *Preprint*, arXiv:2202.05144.

Deng Cai, Shipeng Yu, Ji-Rong Wen, and Wei-Ying Ma. 2003. VIPS: a vision-based page segmentation algorithm. Technical Report MSR-TR-2003-79, Microsoft Research.

Deng Cai, Shipeng Yu, Ji-Rong Wen, and Wei-Ying Ma. 2004. Block-based web search. In *Proceedings of the 27th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pages 456–463.

Xingyu Chen, Zihan Zhao, Lu Chen, Danyang Zhang, Jiabao Ji, Ao Luo, Yuxuan Xiong, and Kai Yu. 2021. WebSRC: A dataset for web-based structural reading comprehension. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Jaemin Cho, Debanjan Mahata, Ozan Irsoy, Yujie He, and Mohit Bansal. 2024. M3docrag: Multi-modal retrieval is what you need for multi-page multi-document understanding. *Preprint*, arXiv:2411.04952.

Zhuyun Dai, Vincent Y. Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B. Hall, and Ming-Wei Chang. 2022. Promptagator: Few-shot dense retrieval from 8 examples. *Preprint*, arXiv:2209.11755.

Chao Deng, Jiale Yuan, Pi Bu, Peijie Wang, Zhong-Zhi Li, Jian Xu, Xiao-Hui Li, Yuan Gao, Jun Song, Bo Zheng, and Cheng-Lin Liu. 2025. LongDocURL: a comprehensive multimodal long document benchmark integrating understanding, reasoning, and locating. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1135–1159, Vienna, Austria. Association for Computational Linguistics.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2web: Towards a generalist agent for the web. *Preprint*, arXiv:2306.06070. NeurIPS 2023.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph RAG approach to query-focused summarization. *Preprint*, arXiv:2404.16130. Microsoft.

Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. 2024. Colpali: Efficient document retrieval with vision language models. *Preprint*, arXiv:2407.01449.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *Preprint*, arXiv:2106.09685.

Ting Jiang, Shaohan Huang, Zhongzhi Luan, Deqing Wang, and Fuzhen Zhuang. 2024. Scaling sentence embeddings with large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3182–3196.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Junlong Li, Yiheng Xu, Lei Cui, and Furu Wei. 2022. MarkupLM: Pre-training of text and markup language for visually-rich document understanding. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Xueguang Ma, Sheng-Chieh Lin, Minghan Li, Wenhu Chen, and Jimmy Lin. 2024a. Unifying multimodal retrieval via document screenshot embedding. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, Pan Zhang, Liangming Pan, Yu-Gang Jiang, Jiaqi Wang, Yixin Cao, and Aixin Sun. 2024b. Mmlongbench-doc: Benchmarking long-context document understanding with visualizations. *Preprint*, arXiv:2407.01523.

Yichuan Wang, Zhifei Li, Zirui Wang, Paul Teiletche, Lesheng Jin, Matei Zaharia, Joseph E. Gonzalez, and Sewon Min. 2026. Pixelrag: Web screenshots beat text for retrieval-augmented generation. *Preprint*, arXiv:2606.28344. UC Berkeley / BAIR / Berkeley NLP. Code: github.com/StarTrail-org/PixelRAG.

Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. 2020. LayoutLM: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Shi Yu, Chaoyue Tang, Bokai Xu, Junbo Cui, Junhao Ran, Yukun Yan, Zhenghao Liu, Shuo Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2024. Visrag: Vision-based retrieval-augmented generation on multi-modality documents. *Preprint*, arXiv:2410.10594.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a generalist web agent, if grounded. *Preprint*, arXiv:2401.01614. ICML 2024.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2023. Webarena: A realistic web environment for building autonomous agents. *Preprint*, arXiv:2307.13854. ICLR 2024.

Yilong Zuo, Xunkai Li, Jing Yuan, Qiangqiang Dai, Hongchao Qin, and Ronghua Li. 2026. MAGE-RAG: Multigranular adaptive graph evidence for agentic multimodal RAG in long-document QA. *Preprint*, arXiv:2606.15906. Code: github.com/laonuo2004/MAGE-RAG.

## A Implementation Details

The pipeline is Python, one module per stage of §3: rendering/extraction via Playwright (`dom_extraction.py`); tiling and pruning over Pillow crops (`tiling.py`); graph construction with `networkx` (`graph_builder.py`); the reader VLM via Ollama (`qwen3-vl`, default, no GPU needed) or Hugging Face `transformers` (`Qwen2-VL-7B-Instruct`) for LoRA fine-tuning, every call cached to disk by a hash of (image, prompt, decoding-config) since decoding is greedy, so a retried page never re-pays a call already made (`vlm_client.py`); synthetic QA generation (`qa_generation.py`); hard-negative mining via `scikit-learn` TF-IDF with a dependency-free fallback (`hard_negative_mining.py`); and contrastive LoRA fine-tuning with `peft`/`torch`, target modules discovered by introspecting the loaded model rather than hard-coded (`lora_finetune.py`).

**Oversized-page splitting, composition, and concurrency.** `sectioning.py` implements the `h1`/`h2` split as a pure function over already-extracted elements, unit-testable without Playwright. `corpus_builder.py` separates network-bound fetch from a synchronous core that tiles, decides whether to split, and composes per-section graphs, exposing an `entry_page` on every graph; multi-page (`links_to`) and multi-section (`continues`) composition share the same node-namespacing machinery. QA

generation issues per-tile VLM calls concurrently within a page (a thread pool sized to Ollama's default parallelism); the controller does the same on the query path, since which nodes open depends only on relevance scores computed upfront, never on a VLM response.

**Batch construction and resolution ablation.** `scripts/build_dataset.py` drives the pipeline over a page list with retrying fetches; its unit of work is a *page unit* (a page, or one split section), checkpointed independently to `contrastive_examples.jsonl` – a failure partway through an oversized page costs only the section in flight, motivated by an early run that lost hours of work when the whole page was the retry unit. `image_compression.py` mirrors tile images at a reduced scale under a separate root with identical filenames and relative paths, so switching a training run between resolutions (§3.3) is a single `--images-root` flag.

**Tiling ablation and multi-hop data.** `scripts/grid_baseline.py` reimplements the blind grid as a module independent of `tiling.py`; `scripts/dom_vs_grid_experiment.py` re-renders each page, builds both tilings, and ranks existing questions against both by TF-IDF (§5). `src/multihop_qa.py` builds cross-page examples from HotpotQA (Yang et al., 2018) records streamed via the `datasets` library; `scripts/build_multihop_dataset.py` reports unreachable-article vs. unmatched-sentence drops separately. Positive-tile attribution uses stopword-filtered token overlap rather than exact substring matching, since the latter rarely survives a decade of article edits since HotpotQA's 2017 source dump.

**Demonstration and tests.** A notebook runs every stage end-to-end with interactive `pyvis` graph visualizations. Dependency-light unit tests cover the geometric pipeline, graph construction and splitting, hard-negative mining (lexical and VLM-judge, stub client), the VLM cache and call concurrency, and resolution compression, all on synthetic fixtures needing no network, VLM, or GPU. Playwright-backed tests check the text-flow bounding-box measurement (floated-sidebar fixture) and the full geometric pipeline end-to-end (DOM extraction through XML export) against a bundled local HTML fixture.

## B Hyperparameters

Every hyperparameter referenced in §3.1–§3.3, in one place.

| Parameter | Value |
|---|---|
| Viewport / max tile width | 2048 px |
| Max tile height (patching) | 2048 px |
| Min tile height (merging) | 80 px |
| Row-grouping tolerance | 20 px |
| Nesting tolerance | 1 px |
| Overlap-grouping threshold | 15% of smaller area |
| Link-bearing tiles kept ($k$) | 3 |
| Oversize-split threshold | 10× max tile height |
| Mined hard negatives ($n$) | 2 |
| Semantic-neighbor threshold | 0.2 cosine |
| Semantic neighbors / tile (max) | 3 |
| Downscale factor (ablation) | 0.5 |
| LoRA $r$ / $\alpha$ / dropout | 16 / 32 / 0.05 |
| InfoNCE temperature $\tau$ | 0.05 |

Table 5: All pipeline hyperparameters (§3.1–§3.3). Each is a single documented, centralized value, not swept; see §6.

## C Compute budget for contrastive fine-tuning

Training (ablation (b)) ran on a rented 80 GB A100/H100-class card, batch size 4, $n$=2 hard negatives, 3 epochs over the 1,339 examples of Table 4. The batched embedding path (`lora_finetune.py`, default) first hit `torch.OutOfMemoryError` on a routine `dropout` call with 79.16 of 79.25 GiB already in use – genuine peak usage, not fragmentation, since our largest tiles (2048 × 2048 px) push a large vision-token count through the batch jointly. `--gradient-checkpointing` (recomputing activations on the backward pass, ∼20–30% slower) resolved it and should be treated as the default starting point at this tile resolution on an 80 GB card, not a fallback for smaller ones only.

## D Tiling ablation: cost detail

Per-tile breakdown behind the pixel-budget figure reported in §5 (ablation (a)).

## E Additional figures

| | Tiles/page | Px/tile | Total px/page |
|---|---|---|---|
| DOM | 24.0 | 153,510 | 3,684,240 |
| Grid | 4.8 | 2,057,956 | 9,946,785 |

Table 6: Tiling cost, same 18 pages as Table 1. DOM-guided tiling produces more, smaller tiles, yet a smaller *total* pixel budget per page (2.7× less) – a direct consequence of the text-flow bounding-box measurement of §3.1, which crops text tiles to the width their content actually occupies instead of the full page width every grid band uses regardless of content. If image tokens scale with pixel area, as they do for `Qwen2-VL`'s vision encoder, DOM-guided tiling need not trade accuracy for token cost against a blind grid.

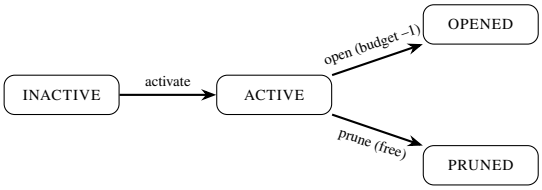


Figure 3: The controller's state machine (§3.4). Opening a node consumes budget and may activate inactive neighbors (`reading_order`, or across a document boundary via `links_to`/`continues`); pruning is free, applied to any node still ACTIVE once budget is exhausted.

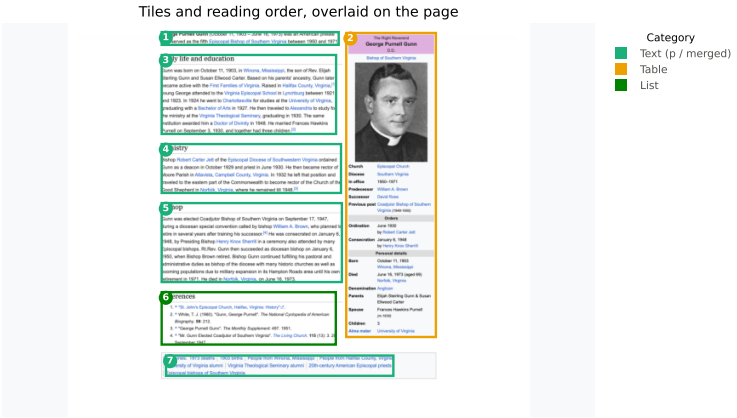


Figure 4: DOM-guided tiles (colored by content category, numbered in reading order) overlaid on the rendered page for `en.wikipedia.org/wiki/George_P._Gunn`. Tile boundaries follow the DOM's own element boundaries: the infobox table (tile 2) is never split mid-row and stays disjoint from the body text and reference list that visually share its column range (tiles 1, 3–6), each section heading is paired with its first content block – a paragraph for tiles 3–5, the reference list itself for tile 6 – rather than emitted alone, and the trailing category footer (tile 7) is merged into its own tile rather than left as a flood of near-empty fragments.

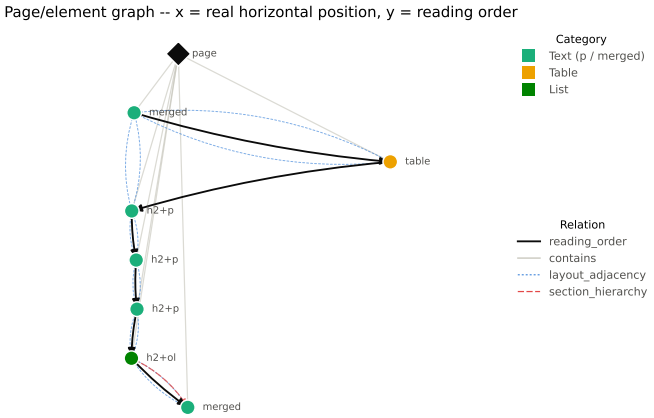


Figure 5: Page/element graph for the same page as Figure 4, laid out by real horizontal position ($x$) and reading-order rank ($y$). `reading_order` (solid black) and `contains` (light gray) are the two relations MAGE-RAG also has; `layout_adjacency` (dotted blue) and `section_hierarchy` (dashed red) are the two we add. Most headings here are paired with their content into one tile (§3.1), so `section_hierarchy` shows only where trailing non-heading content (the category footer) attaches to the last open section, rather than between the (sibling, not nested) section headings themselves.

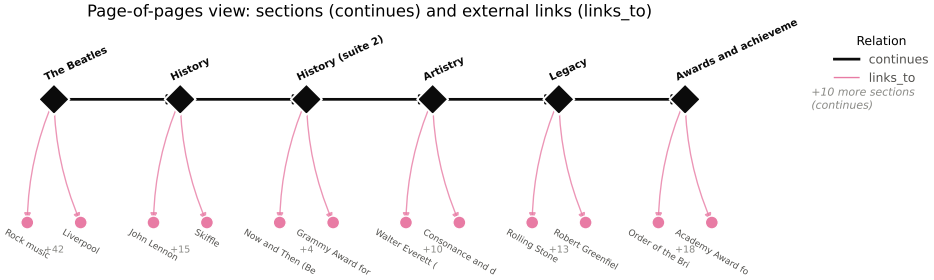


Figure 6: Page-level view of the graph for `en.wikipedia.org/wiki/The_Beatles` (278 DOM elements, split into 16 sections at `h1`/`h2` boundaries – only the first 6 are shown for legibility). Each diamond is a `page` node, one per section; `continues` edges (black) chain them in document order; `links_to` edges (pink) reach external pages, collapsed here from their true element-level source (§3.2) to stay at page granularity. The leftmost node is the entry point.

```
<page url="..." title="...">
 <element id="tile_0000" tag="merged"
      state="inactive">...</element>
 <element id="tile_0003" tag="p"
      state="opened">Gunn was born
  on October 11, 1903...</element>
 <element id="tile_0004" tag="h2+p"
      state="inactive">...</element>
 ...
</page>
```

Figure 7: Excerpt of the XML rendering of a page graph after the evidence controller has run on the query "When and where was George Purnell Gunn born?". Only the paragraph tile carrying the birth date/place has been opened; the rest remains inactive or pruned.